

Практическая работа №4.

«ОБУЧАЮЩИЕ, ВАЛИДАЦИОННЫЕ И ТЕСТОВЫЕ НАБОРЫ ДАННЫХ В МАШИННОМ ОБУЧЕНИИ»

В большинстве задач машинного обучения (ML) с учителем рекомендуется разделять данные на три независимых набора: **обучающий набор, тестовый набор и набор для валидации** — это передовая практика в ML, обеспечивающая хорошую обобщающую способность моделей. Обучающие данные обучают модель, валидационные данные дорабатывают её, а тестовый набор данных предоставляет непредвзятую оценку на неизвестных данных. Разделение на обучающую, проверочную и тестовую выборки имеет фундаментальное значение для разработки надежных и устойчивых моделей ML.

Для обеспечения обобщаемости алгоритма ML крайне важно разделить набор данных на три сегмента: обучающий набор, проверочный набор и тестовый набор. Это позволит реалистично измерить производительность вашей модели, гарантируя, что набор данных, используемый для обучения модели, и набор данных, используемый для ее оценки, являются разными

1). Набор обучающих данных (Train)

Выборка данных, использованная для построения модели. Набор данных, который мы используем для аппроксимации непредвзятой точности нашей модели в реальных условиях. Другими словами, модель анализирует данные из обучающего набора и учится на них, чтобы напрямую улучшать свои параметры.

Для максимальной эффективности модели обучающий набор данных должен быть (i) достаточно большим, чтобы давать значимые результаты (но не слишком большим, чтобы модель не переобучалась), и (ii) репрезентативным для всего набора данных. Это позволит обученной модели прогнозировать любые ранее неизвестные данные, которые могут появиться в будущем.

2). Набор данных для валидации, или проверки (Validation)

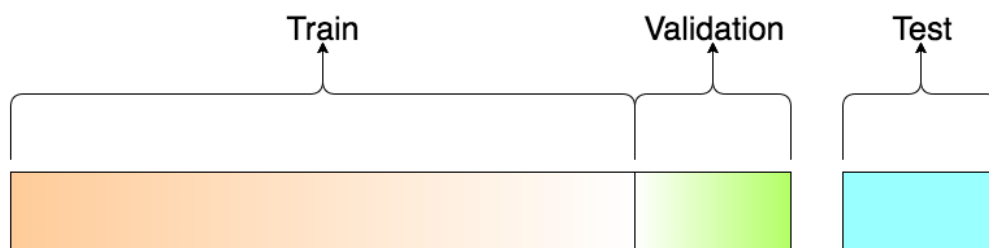
Выборка данных используется для обеспечения непредвзятой оценки соответствия модели обучающему набору данных при настройке гиперпараметров модели. Оценка становится более предвзятой по мере включения в конфигурацию модели навыков, полученных на валидационном наборе данных. Набор данных, который мы используем для оценки производительности нашей модели при различных типах моделей и выборе гиперпараметров.

Набор данных для валидации — набор данных, используемый для оценки и тонкой настройки модели ML в процессе обучения, помогающий оценить производительность модели и внести необходимые корректировки.

Оценивая обученную модель на валидационном наборе данных, мы получаем представление о ее способности к обобщению на неизвестные данные. Эта оценка помогает выявить потенциальные проблемы, такие как переобучение, которое может существенно повлиять на производительность модели в реальных условиях.

3) Тестовый набор данных (Test)

Выборка данных, используемая для объективной оценки соответствия окончательной модели обучающему набору данных. Набор данных, который мы используем для аппроксимации непредвзятой точности нашей модели в реальных условиях.



Разделение набора данных на два подмножества — хорошая идея, но не совсем разумная. Вы можете значительно снизить вероятность переобучения, разделив набор данных на три подмножества. Используйте проверочный набор для оценки результатов, полученных на обучающем наборе. Затем используйте тестовый набор для повторной проверки оценки после того, как модель «пройдёт» проверку на проверочном наборе. На следующем рисунке показан этот рабочий процесс:

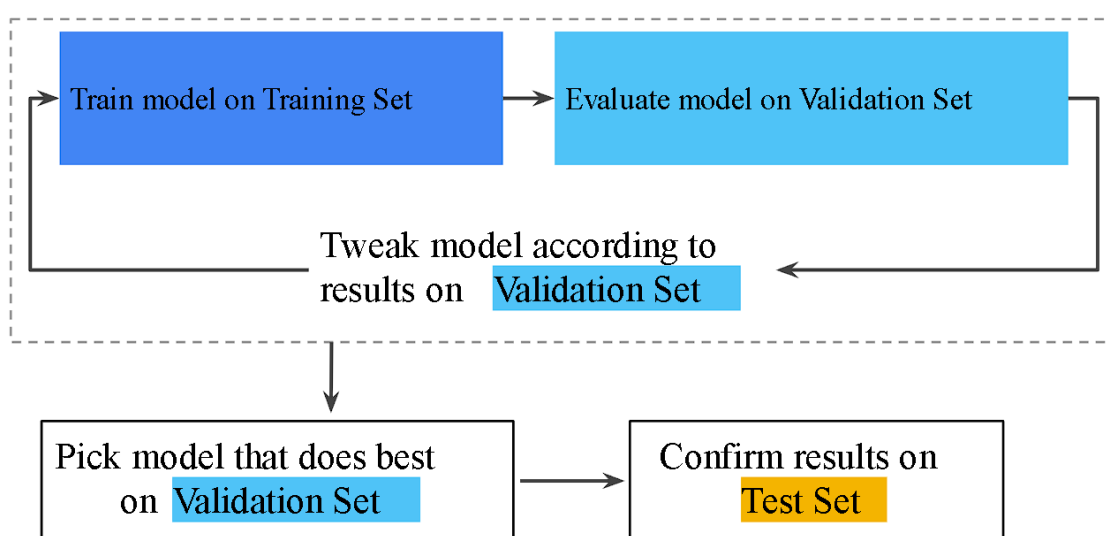


Рисунок 1.

В целом, этапы следующие:

- 1). Разделить имеющиеся данные на обучающую, валидационную и тестовую выборки.
- 2). Выбрать алгоритм и параметры обучения.
- 3). Обучить модель, используя обучающую выборку.
- 4). Оценить модель, используя валидационную выборку.
- 5). Повторить шаги 2–4, используя разные алгоритмы и параметры обучения.
- 6). Выбрать лучшую модель и обучить ее, используя данные из обучающей и валидационной выборок.
- 7). Оценить эту окончательную модель, используя тестовую выборку.

Как разделить

Не существует общепринятого правила для определения того, какие пропорции данных следует выделить для трех выборок (обучающей, валидационной и тестовой). Общий критерий заключается в наличии достаточного количества данных в валидационной и тестовой выборках для надежной оценки риска прогностических моделей.

Популярные варианты: 60-20-20, 70-15-15, 80-10-10 (рис. 2).

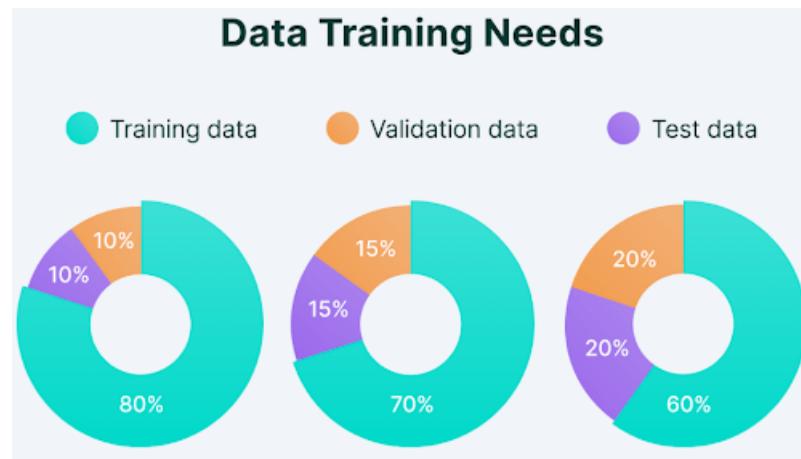


Рисунок 2. Различные распределения данных для обучения, проверки и тестирования

Могут быть и такие варианты разделения данных.

	train split	validation split	test split
proportion of original dataset	50% to 70%	20% to 30%	10% to 20%

ЗАДАНИЕ 1.

Изучить примеры 1 и 2, в которых показано разделение данных на обучающие, валидационные и тестовые наборы данных.

Пример 1: прогнозирование инфляции

Разбить данные на обучающую, валидационную и тестовую выборки. Будем использовать набор данных, в котором:

- Результатом y_t является месячная инфляция HICP в еврозоне;
- Входной вектор x_t включает ежемесячные фиктивные переменные (для учета сезонности) и более 100 макроэкономических переменных в реальном времени.

```

import numpy as np # Numpy is a Matlab-like package for array manipulation and linear algebra
import pandas as pd # Pandas is a data-analysis and table-manipulation tool
import urllib.request # Urllib will be used to download the dataset

# Load the output variable with pandas (download with urllib if not downloaded previously)
remoteAddress = 'https://www.statlect.com/ml-assets/y_hicp.csv'
localAddress = './y_hicp.csv'
try:
    y = pd.read_csv(localAddress, header=None)
except:
    urllib.request.urlretrieve(remoteAddress, localAddress)
    y = pd.read_csv(localAddress, header=None)
y = y.values # Transform y into a numpy array

# Print some information about the output variable
print('Class and dimension of output variable:')
print(type(y))
print(y.shape)

```

```

Class and dimension of output variable:
<class 'numpy.ndarray'>
(270, 1)

```

```

# Load the input variables with pandas
remoteAddress = 'https://www.statlect.com/ml-assets/x_hicp.csv'
localAddress = './x_hicp.csv'
try:
    x = pd.read_csv(localAddress, header=None)
except:
    urllib.request.urlretrieve(remoteAddress, localAddress)
    x = pd.read_csv(localAddress, header=None)
x = x.values

# Print some information about the input variables
print('Class and dimension of input variables:')
print(type(x))
print(x.shape)

```

```

Class and dimension of input variables:
<class 'numpy.ndarray'>
(270, 113)

```

Используйте Scikit-learn для разделения на обучающую, валидационную и тестовую выборки (60-20-20). Поскольку Scikit-learn выполняет разделение выборки только на две подвыборки, то сначала создаем обучающую выборку, а затем разделяем оставшиеся наблюдения на валидационные и тестовые.

```

from sklearn.model_selection import train_test_split

x_train, x_val_test, y_train, y_val_test = train_test_split(x, y, test_size=0.4, random_state=1)

x_val, x_test, y_val, y_test = train_test_split(x_val_test, y_val_test, test_size=0.5, random_state=1)

print(x_train.shape[0], x_val.shape[0], x_test.shape[0])

```

Оценить и протестировать линейную регрессию со всеми входными данными.

Здесь показано, что модель линейной регрессии с большим количеством параметров, оцениваемая методом наименьших квадратов, переобучается на обучающем наборе данных и демонстрирует неудовлетворительные результаты на валидационном наборе данных. Пока не используем тестовый набор данных, потому что собираемся опробовать другие модели и затем выбрать лучшую.

```
from sklearn import linear_model # Linear regression
from sklearn.metrics import mean_squared_error, r2_score # MSE and R squared

# Create linear regression object
lr0 = linear_model.LinearRegression()

# Train the model using the training set
lr0.fit(x_train, y_train)

# Make predictions on the training and validation sets
y_train_pred = lr0.predict(x_train)
y_val_pred = lr0.predict(x_val)

# Print empirical risk on both sets
print('MSE on training set:')
print(mean_squared_error(y_train, y_train_pred))
print('MSE on validation set:')
print(mean_squared_error(y_val, y_val_pred))
print('')

# Print R squared on both sets
print('R squared on training set:')
print(r2_score(y_train, y_train_pred))
print('R squared on validation set:')
print(r2_score(y_val, y_val_pred))
MSE on training set:
0.014389920273540512
MSE on validation set:
0.1679610389980635

R squared on training set:
0.9084867354573687
R squared on validation set:
0.2894515759305918
```

Пример 2: изучите разделение данных с их последующим сохранением в файлы *csv (см. файл 2_data_splitting.ipynb).

ЗАДАНИЕ 2.

1. На основе данных и модели, использованных в **Практической работе 3**, решить задачу регрессии, используя разделение данных на **train**, **val** и **test** в программе и в отдельных файлах (trainR.csv, valR.csv и testR.csv).
2. Дополнить решение задачи регрессии. Реализуйте модель регрессии с

использованием **стохастического градиентного спуска** – класс **SGDRegressor** из библиотеки Scikit-learn. Оформите построение моделей в виде конвейера (Pipeline), включив в него необходимые этапы предобработки данных (например, стандартизацию признаков).

https://scikit-learn.org/stable/modules/linear_model.html#stochastic-gradient-descent-sgd

<https://scikit-learn.org/stable/modules/sgd.html>

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDRegressor.html#sklearn.linear_model.SGDRegressor

Примеры

```
>>> import numpy as np
>>> from sklearn.linear_model import SGDRegressor
>>> from sklearn.pipeline import make_pipeline
>>> from sklearn.preprocessing import StandardScaler
>>> n_samples, n_features = 10, 5
>>> rng = np.random.RandomState(0)
>>> y = rng.randn(n_samples)
>>> X = rng.randn(n_samples, n_features)
>>> # Always scale the input. The most convenient way is to use a pipeline.
>>> reg = make_pipeline(StandardScaler(),
...                     SGDRegressor(max_iter=1000, tol=1e-3))
>>> reg.fit(X, y)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('sgdregressor', SGDRegressor())])
```

3. **Обучение и оценка.** Обучите модель на обучающей выборке. Используйте валидационную выборку для подбора гиперпараметров (например, выбор скорости обучения, штрафа, количества итераций). Оцените качество итоговой модели на тестовой выборке.

Выведите метрики регрессии: **среднеквадратичную ошибку (MSE)** и **коэффициент детерминации (R^2)** для всех трёх выборок (train, val, test).

4. Создать таблицу результатов (объект DataFrame), в которой укажите названия регрессоров и значения метрик качества на обучающей и тестовой выборках. Обеспечьте нужное количество знаков после запятой (см. Образец 1).

Образец 1

	Train Data					Val Data					Test Data
Регрессор	R^2	MSE	RMSE	MAE	MAPE	R^2	MSE	RMSE	MAE	MAPE	...
Linear Regression	0.XX	0.XXXX	0.XXXX	0.XXXX	0.XXXX	0.XX	0.XXXX	0.XXXX	0.XXXX	0.XXXX	...
...											

5. Написать вывод о выполненной Практической работе №4 и сравнить результаты данной работы с результатами, полученными в Практической работе №3.